

Pregunta De Implementación en Código (12 ptos.)

La universidad UTEC necesita promover el reciclaje entre estudiantes. Se requiere desarrollar EcoRecicla UTEC, un servicio REST que gestione puntos de reciclaje, registro de aportes de materiales reciclables y un sistema de recompensas. Su implementación debe usar arquitectura en 3 capas (Controller, Service, Repository), DTOs para entrada/salida, manejo correcto de excepciones y seguridad JWT.

Entidades

El sistema necesita manejar los siguientes objetos de dominio:

User

```
Java
id: Long (autogenerado)
username: String (único, obligatorio)
email: String (único, obligatorio)
password: String (obligatorio, encriptado)
role: String (default: ROLE_STUDENT)
```

RecyclingCenter

```
Java
id: Long (autogenerado)
name: String (obligatorio)
location: String (obligatorio)
materialTypes: String (comma-separated: PLASTIC, PAPER, GLASS, METAL, ELECTRONIC)
status: String (ACTIVE, INACTIVE, FULL)
capacity: Integer (kg, ≥ 0)
currentLoad: Integer (kg, ≥ 0)
```

Contribution

```
Java
id: Long (autogenerado)
userId: Long (referencia a User)
centerId: Long (referencia a RecyclingCenter)
materialType: String (PLASTIC, PAPER, GLASS, METAL, ELECTRONIC)
weight: Double (kg, > 0)
```

```
contributedAt: ZonedDateTime
pointsEarned: Integer (calculado según tabla de conversión)
status: String (PENDING, VERIFIED, REJECTED)
```

Reward

```
Java
id: Long (autogenerado)
userId: Long (referencia a User)
rewardType: String (COFFEE, SNACK, BOOK, MERCHANDISE)
pointsCost: Integer (> 0)
redeemedAt: ZonedDateTime
status: String (PENDING, DELIVERED, CANCELLED)
```

Endpoints

1. Registro de Usuario: POST /auth/register

```
JSON
// Request DTO
{
  "username": "juan.eco",
  "email": "juan@utec.edu.pe",
  "password": "Recicla2025"
}

// Response DTO
{
  "id": 1,
  "username": "juan.eco",
  "email": "juan@utec.edu.pe",
  "totalPoints": 0
}
```

Status Code: Created

Validaciones:

- username/email únicos
- password $\geq$ 8 caracteres
- email debe terminar en @utec.edu.pe

2. Autenticación: POST /auth/login

```
JSON
// Request DTO
{
  "username": "juan.eco",
  "password": "Recicla2025"
}

// Response DTO
{
  "token": "eyJhbGciOiJIUzI1NiIs... ",
  "expiresIn": 3600
}
```

Status Code: OK

3. Crear Centro de Reciclaje: POST /centers

```
JSON
// Request DTO
{
  "name": "Centro Pabellón A",
  "location": "Pabellón A - Primer Piso",
  "materialTypes": "PLASTIC,PAPER,GLASS",
  "capacity": 500
}

// Response DTO
{
  "id": 1,
  "name": "Centro Pabellón A",
  "location": "Pabellón A - Primer Piso",
  "materialTypes": "PLASTIC,PAPER,GLASS",
  "capacity": 500,
  "currentLoad": 0,
  "status": "ACTIVE"
}
```

Headers:

- **Authorization:** Bearer {{token}}

- **Content-Type:** application/json

Status Code: Created

Validaciones:

- name no debe estar vacío
- location no debe estar vacío
- capacity ≥ 100
- materialTypes debe contener al menos un tipo válido de la lista: PLASTIC, PAPER, GLASS, METAL, ELECTRONIC
- materialTypes debe estar en formato comma-separated sin espacios

Lógica:

- currentLoad se inicializa en 0
- status se inicializa en ACTIVE

4. Listar Centros de Reciclaje: GET /centers

```
JSON
// Response DTO
{
  "content": [
    {
      "id": 1,
      "name": "Centro Pabellón A",
      "location": "Pabellón A - Primer Piso",
      "materialTypes": "PLASTIC,PAPER,GLASS",
      "availableCapacity": 450,
      "status": "ACTIVE"
    },
    {
      "id": 2,
      "name": "Centro Biblioteca",
      "location": "Biblioteca Central",
      "materialTypes": "PAPER,ELECTRONIC",
      "availableCapacity": 200,
      "status": "ACTIVE"
    }
  ],
  "page": 0,
  "size": 10,
  "totalElements": 5
}
```

Headers:

- **Authorization:** Bearer {{token}}

Query Parameters:

- **status:** all | active | full (default: active)
- **materialType:** PLASTIC | PAPER | GLASS | METAL | ELECTRONIC (opcional)
- **page:** 0 (default)
- **size:** 10 (default)

Status Code: OK

Lógica:

- **active:** centros con status ACTIVE
- **full:** centros con status FULL
- **all:** todos los centros
- Si materialType especificado, filtrar centros que acepten ese material

5. Registrar Aporte: POST /contributions

```
JSON
// Request DTO
{
  "centerId": 1,
  "materialType": "PLASTIC",
  "weight": 2.5
}

// Response DTO
{
  "id": 10,
  "userId": 1,
  "centerId": 1,
  "centerName": "Centro Pabellón A",
  "materialType": "PLASTIC",
  "weight": 2.5,
  "pointsEarned": 25,
  "contributedAt": "2025-10-05T10:30:00-05:00",
  "status": "PENDING"
}
```

Headers:

- **Authorization:** Bearer {{token}}
- **Content-Type:** application/json

Status Code: Created

Tabla de conversión de puntos:

- **PLASTIC:** 10 puntos/kg
- **PAPER:** 5 puntos/kg
- **GLASS:** 8 puntos/kg
- **METAL:** 15 puntos/kg
- **ELECTRONIC:** 20 puntos/kg

Validaciones:

- $\text{weight} > 0 \text{ y } \leq 50 \text{ kg}$
- Centro debe aceptar ese tipo de material
- Centro no debe estar FULL

Lógica:

- Actualizar currentLoad del centro
- Si $\text{currentLoad} \geq \text{capacity}$, cambiar status del centro a FULL
- userId se obtiene del token JWT
- Calcular pointsEarned según tabla de conversión

6. **Canjear Recompensa:** POST /rewards/redeem

JSON

// Request DTO

```
{  
  "rewardType": "COFFEE"  
}
```

// Response DTO

```
{  
  "id": 5,  
  "userId": 1,  
}
```

```
"rewardType": "COFFEE",
"pointsCost": 50,
"remainingPoints": 100,
"redeemedAt": "2025-10-05T11:00:00-05:00",
"status": "PENDING",
"code": "RF-5-20251005"
}
```

Headers:

- **Authorization:** Bearer {{token}}
- **Content-Type:** application/json

Status Code: Created

Catálogo de recompensas:

- **COFFEE:** 50 puntos
- **SNACK:** 30 puntos
- **BOOK:** 200 puntos
- **MERCHANDISE:** 300 puntos

Lógica:

- Validar que el usuario tenga suficientes puntos
- Descontar puntos del total acumulado del usuario
- Generar código único de canje formato: "RF-{rewardId}-{YYYYMMDD}"
- userId se obtiene del token JWT

Excepciones

La implementación debe contemplar el uso de por lo menos las siguientes excepciones:

- **InvalidEmailDomainException** → **400 Bad Request**
 - Caso: **POST /auth/register** con email que no termina en @utec.edu.pe
- **RecyclingCenterNotFoundException** → **404 Not Found**
 - Caso: **POST /contributions** con centerId inexistente
- **CenterFullException** → **409 Conflict**
 - Caso: **POST /contributions** cuando el centro tiene status FULL
- **InvalidMaterialTypeException** → **400 Bad Request**

- Caso: **POST /contributions** con material no aceptado por el centro
- **InsufficientPointsException → 400 Bad Request**
 - Caso: **POST /rewards/redeem** sin puntos suficientes
- **InvalidWeightException → 400 Bad Request**
 - Caso: **POST /contributions** con $\text{weight} \leq 0$ o > 50